

Hands-On Lab: Multi-Agent Orchestration in Practice

Domain Copilot (D3T5) — Technical Training Session

In this session, you will work directly against the backend repository to explore, test, and debug a production-ready **Human-in-the-Loop (HITL) Multi-Agent Workflow** built with **FastAPI**, **LangGraph**, and **PostgreSQL**.

Step 0: Clone the Repository & Setup

Before diving into the exercises, let's get your local environment up and running.

1. Clone the repository:

Bash

```
git clone <repository-url>
cd <repository-directory>
```

2. Set up your Python virtual environment:

```
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

3. Configure your environment variables:

- Copy the example environment file:

```
cp .env.example .env
```

-
- Open .env and verify your local PostgreSQL connection string .

=====

Part 1 — Spin Up & Verify (10 min)

Goal: Ensure your local database, containers, and architecture test suite are fully operational.

1.1 Bring Up the Stack

Run Docker Compose to spin up your local database and API services:

```
docker compose up --build
```

Expected Output: Both api-1 and postgres-1 containers report healthy. The API logs confirm Uvicorn is running on http://0.0.0.0:8000.

1.2 Run the Architecture Boundary Test

Run the architectural validation test suite:

```
pytest backend/tests/test_architecture_boundaries.py -v
```

Expected Output: All tests pass, verifying strict separation of concerns—confirming domain/ and application/ contain zero imports of langgraph, fastapi, openai, sqlalchemy, or psycopg2.

- **Checkpoint Question:** Open `backend/infrastructure/orchestration/copilot_graph.py` and `backend/application/agents/standards_mapper.py` side by side. Which file is allowed to import `LangGraph`, and why does the other file remain completely agnostic to it?

1.3 Run the Full Test Suite

Run the complete testing baseline:

```
pytest backend/tests/ -v
```

Expected Output: All baseline tests pass. Take note of the total test count—you will compare this number again after completing Part 3!

=====

Part 2 — Execute & Inspect via Minimal UI (15 min)

- **Goal:** *Run the end-to-end multi-agent workflow, observe the human review pause gate visually in the browser, and approve execution via the UI dashboard.*

Test Accounts & Credentials

Use the following credentials to log into the UI dashboard (<http://localhost:3000>):

- **Standard Instructor Account (Review Queue & Approvals)**
 - Email: `instructor@example.com`
 - Password: `password123`
- **Lead Instructor Account (Full Administrative Access)**
 - Email: `lead@example.com`
 - Password: `password123`

2.1 Trigger the Workflow

Run the demo script in your terminal to initialize the agents (`standards_mapper -> module_outline_generator -> assessment_generator`):

```
python -m backend.scripts.run_hitl_demo
```

Expected Output:

Plaintext

```
Initializing Domain Copilot HITL Workflow Demo ...
standards_mapper -> module_outline_generator -> assessment_generator ...
Paused before human_review. thread_id = demo-thread-001
```

- **Checkpoint Question:** The graph has stopped running. Is the Python process that ran this script still alive? Is the workflow state still alive? Where does that state actually live right now?

2.2 Open the Minimal UI Dashboard

1. Open your browser and navigate to <http://localhost:8000/static/index.html>
2. Log in using one of the test accounts provided above and navigate directly to the Review Queue.

2.3 Inspect & Approve Visually

1. Locate demo-thread-001 in the queue and click on it to inspect the generated Competency Gap Report.
2. Click Approve (with an optional instructor comment).
3. Watch the system seamlessly resume execution and transition to the publication step.

2.4 Confirm the Persistence Write

Verify via the terminal that the curriculum record was safely committed to the database:

```
python -c "  
    from backend.infrastructure.db.repositories.published_curriculum_repository  
        import SQLAlchemyPublishedCurriculumRepository  
        # Verify repository wiring per README  
        print(repo.get_by_thread_id('demo-thread-001')) "  
"
```

Expected Output: A valid PublishedCurriculum record object (not None).

- Checkpoint Question: What part of the application logic would have blocked this record from being written if you had clicked Reject instead of Approve?

Part 3 — Break & Fix (25 min)

- *Goal: Replicate real production bugs in the code and test suites, observe the failures, and apply proper architectural fixes.*

3.1 Exercise A: Break the Reducer (audit_trail Overwrite Bug)

3.1 Exercise A: Break the Reducer (`audit_trail` Overwrite Bug)

- Break it: In `backend/infrastructure/orchestration/copilot_graph.py`, temporarily change:

Python



```
audit_trail: Annotated[list[dict[str, Any]], operator.add]
```

to a plain type annotation:

Python



```
audit_trail: list[dict[str, Any]] | None
```

- Test it: Run the HITL workflow again, approve the task, and check the `audit_trail` in the final state.
- Expected Failure: Only the final node's audit entry survives; earlier entries are silently overwritten.
- Fix it: Restore the `Annotated[..., operator.add]` reducer. Re-run and verify the complete history is preserved.

3.2 Exercise B: Break the Checkpointer (Non-Serializable State)

3.2 Exercise B: Break the Checkpointer (Non-Serializable State)

- **Break it:** In `backend/tests/test_langgraph_orchestrator.py`, locate a test constructing initial state and inject a non-serializable object:

Python



```
from unittest.mock import Mock
state["debug_object"] = Mock()
```

- **Test it:** Run that test against a real `PostgresSaver` checkpointer.
- **Expected Failure:**

Plaintext



```
TypeError: Type is not msgpack serializable: Mock
```

- **Fix it:** Remove the `Mock()` field and confirm the test passes successfully.

3.3 Exercise C: Break the State Machine (Illegal Transition Bypass)

3.3 Exercise C: Break the State Machine (Illegal Transition Bypass)

- **Break it:** In `backend/infrastructure/db/repositories/review_task_repository.py`, locate `update_status` and temporarily comment out the `can_transition(...)` guard check.
- **Test it:** Create a review task (defaulting to `pending`), then call `update_status(task_id, "approved")` directly—completely skipping assignment.
- **Expected Failure:** The illegal transition succeeds silently, jumping straight from `pending` to `approved` without an assigned reviewer.
- **Fix it:** Restore the transition guard. Re-run the script and confirm it raises `ValueError: Illegal state transition from pending to approved`.

- **Checkpoint Question:** Given this strict guard exists, why does `process_review_decision()` in `human_review_service.py` still succeed when a user approves a task straight from the UI without explicitly calling an assign endpoint first?

3.4 Exercise D: Break the Provider Abstraction (Cross-Provider Bug)

3.4 Exercise D: Break the Provider Abstraction (Cross-Provider Bug)

- **Break it:** In `backend/application/agents/standards_mapper.py`, locate the `self._llm_provider.complete(...)` call and confirm it passes `options={"temperature": 0.0, ...}`. Set `LLM_PROVIDER=openai` in your `.env` file (with a valid API key) and run the HITL demo.
- **Expected Failure:** A `TypeError` thrown by the OpenAI SDK because `options` is not a recognized native parameter.
- **Fix it:** Remove the `options={...}` kwarg (or map it to a parameter supported by both adapters). Re-run with `LLM_PROVIDER=openai` to verify clean completion.

Part 4 — Stretch Challenges (Optional Advanced Tasks)

- **Stretch 1 (Escalation Coverage):** Write a new test verifying that overdue `in_review` tasks are correctly handled by the escalation worker, ensuring historical bugs don't regress.
- **Stretch 2 (Write an ADR):** Draft an Architectural Decision Record (`docs/adr/002-hitl-state-management.md`) explaining why publication as a separate table vs. a `ReviewTask` status was chosen, detailing at least one rejected alternative.
- **Stretch 3 (Design a Fourth Agent):** Design a new agent (e.g., `RubricCalibrator`) that runs after `assessment_generator` to validate items against standard writing rules before populating the review queue. Map out where it fits in the graph and what state fields it reads/writes.
- **Stretch 4 (Indirect Prompt Injection):** Run `pytest backend/tests/test_indirect_prompt_injection.py -v`. Examine the poisoned

document it ingests and analyze why this security test couldn't exist prior to multi-agent pipeline implementation.

Answer Key & Explanations

3.1 — Reducer Fix Explanation

- **The Bug:** Without an explicit reducer, LangGraph's default state merging behavior for TypedDict fields is to wholesale overwrite the previous value with whatever the latest node returned.
- **The Fix:** Declare the field using `Annotated[list[dict[str, Any]], operator.add]`.
- **Why it works:** `operator.add` tells LangGraph's state engine to *concatenate* lists returned by successive nodes rather than replacing them, preserving a complete, append-only history for your `audit_trail`.

3.2 — Serialization Fix Explanation

- **The Bug:** LangGraph checkpointers (like `PostgresSaver`) serialize state payloads using high-performance protocols like `msgpack`. Objects like `Mock()`, lambdas, file handles, or runtime memory references cannot be serialized and throw a `TypeError`.
- **The Fix:** Remove the mock object and ensure all state fields store pure data, primitive types, or serializable dataclasses.
- **Why it works:** Checkpointers require persistent, stateless, purely serializable snapshots so that paused workflows can be safely reloaded from the database hours or days later.

3.3 — Transition Guard Explanation

- **The Bug:** Commenting out the `can_transition(...)` check removes the state machine's safety invariants, allowing review tasks to skip required lifecycle steps (like jumping straight from pending to approved without ever being claimed).
- **The Fix:** Restore the `can_transition()` guard check inside `update_status`.
- **Why it works:** The UI service works seamlessly because it explicitly checks if the task is pending and auto-assigns it (`self._review_tasks.assign(...)`) right before updating, satisfying the legal path (pending -> in_review -> approved) programmatically under the hood.

3.4 — Provider Abstraction Fix Explanation

- **The Bug:** Passing adapter-specific kwargs like `options={"temperature": 0.0}` breaks the provider abstraction when switching between different LLMs (e.g., OpenAI vs. Ollama), because the OpenAI SDK does not recognize `options` as a native parameter.
- **The Fix:** Remove `options={...}` from the `complete()` call entirely.
- **Why it works:** Configuration parameters must be passed through standardized kwargs that are explicitly translated by each adapter implementation, preserving the "swap provider via config with zero code change" architecture guarantee.



Submission Checklist

- ☐ **Part 1:** Architecture boundary test passes; full test suite passes.
- ☐ **Part 2:** One full approve-to-publish cycle completed via the minimal UI dashboard; database record confirmed.
- ☐ **Part 3:** All four break/fix exercises completed, with failing outputs noted.
- ☐ **Part 4:** At least one stretch challenge attempted.